SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floating-point computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator **Faculty Incharge**

Computer Architecture - CSA1214
SLOT - A

B. Sivarangini
192521375
B. Tech IT

8/8/24

Assessment Tool / CO2-AT2
Debugging & Optimisation Task

1) 120 + 50 (8-bit)

120 = 01111000 ; 50 = 00110010

120 } ⇒ 01111000
50 } 00110010
MSB (-ve) 10101010

2's complement ⇒ 10101010
is 01010101
+1
10101011 = +86
10101010 = -86

Actual answer = 170

Processor O/P ≠ Actual Answer

120 and 50 are +ve But answer is (-ve)

The signed 8-bit range -128 to +127

But, 170 > +127

Cause of Error:

As a result, the overflow is ignored, and an incorrect negative value is produced.

Debugging:

Instead of 8-bit integers, use 16-bit

So, 120 + 50 = 170

01111000
00110010
10101010$_2$

2) Ripple Carry Adder (RCA)

Take, A = 1011 (11) , B = 0101 (5)

Usual O/P : 11+5 = 16 = 10000

By using RCA:

	C_4	C_3	C_2	C_1	C_0	(Carry - C)
		1	0	1	1	
+		0	1	0	1	
	1	0	0	0	0	

Carry Propagation: = 1+1 = 10, Sum = 0, Carry = 1

Bit . 0 = 1+1=10 . S = 0, C = 1

Bit , 1 = 1+0 + Carry (1) => S = 0, C = 1

Bit , 2 = 0+1+ Carry (1) => S = 0, C = 1

Bit , 3 = 1+0+C(1) => S = 0, c = 1

Final Carry = 1 => ∴ 10000

Debugging: Bit 0 → Carry 0 → Bit 1 → C1 → B2 → C2 → B3 → C3

The carry must ripple through every stage because the final result is obtained

→ As the no. of bits increase (8-bit, 16-bit, 32) the delay also increase

Working of Carry look-Ahead Adder:

I/P → Generate → Propogate → CLA → All carries compute Simultaneously → Final Sum

3) Booth's Algorithm:

$-6 \xrightarrow{(M)} \times (+5) \rightarrow (Q)$

n	A	a	q_0	Action
	0	0101	0	Initialization
4	0	0101	0	A = A-M
	1010	0101	1	ASR
	0011	0010	1	A = A+M
3	0011	0010	0	ASR
	1110	1001	0	A = A-M
2	1110	1001	1	ASR
	0010	0100	1	A = A+M
1	0010	0100	0	
	1110	0010		

$AQ = 11100010_2$

$-6 \times (+5) = -30$ → Hence, Product is Correct

Debugging: (Algorithm)

Incorrect: 1) Bit Pair checking ⇒ Q_0Q → 00, 01, 10, 11 } A+M instead A-M gives incorrect product

2) ASR ⇒ 1110 → 1111; error: 1110 → 0111

3) Sign Extension 1110 → 1111; error: 1110 → 0111

Non-Restoring Division Algorithm:

13 ÷ 3 → (M)

↓

(Q)

n	A	Q	Action
	0000	1101	Initialization
4	0001	1010	LS(A, Q)
	1110	1010	A = A - M
	1110	1010	SQB = 0
3	1101	0100	LS(A, Q)
	0000	0100	A = A + M
	0000	0101	SQB = 1
2	0000	1010	LS(A, Q)
	1101	1010	A = A - M
	1101	1010	SQB = 0
1	1011	0100	LS(A, Q)
	1110	0100	A = A + M
	0001	0100	SQB = 1

Quotient(Q) = 4, Remainder (A) = 1

Debugging : Subtract → (-ve) result

Restore → Continue

Every negative remainder requires an additional restoration (operation)

So, these extra operations increases the execution time.

25) IEEE 754 single Precision :

Eg: 10000000 + 1

Expected result : 100000001

But, obtained result : 1000000

Debugging :

i) $1.0111110101111\ 00001\ 00000 \times 2^{26}$

ii) $1.00000000000000000000 \times 2^{0}$

$1.0 \times 2^{0} \rightarrow 0.00000000000000001 \times 2^{26}$

ii) 1. 01111 010 11110000 100000

+

0. 00000000000 00000000000

= 1.01111101011110000010000

iii) $10.111010101 \rightarrow 1.0111010101 \times 2^{1}$

iv) 1.10101010101010101010

Extra bits

1011

Error caused by :

→ Exponent alignment shifts the small no.

→ Limited 23-bit mantissa cannot store all significant bits.